SkillNavora

MODULE 6 HANDBOOK

Module 6: AI Agents and Intelligent Automation

7 sections, every topic explained with how it works, examples and resources.

From Non-Coder to Enterprise AI Builder: Learn, Build, Deploy, Get Hired and Launch Your AI Business.

Varun Kumar · SkillNavora · skillnavora.in

Edition: November 2026. Links point to official documentation where available; check them for the latest versions.

Financial figures are hypothetical examples, not advice.

Module 6: AI Agents and Intelligent Automation

Every section below explains the concept, how it works, each listed topic in detail, a worked example and where to learn more.

Sections

1. 6.1 AI Agent Fundamentals
2. 6.2 Agent Development Frameworks
3. 6.3 Tool Calling
4. 6.4 MCP: Model Context Protocol
5. 6.5 Multi-Agent Systems
6. 6.6 Human-in-the-Loop
7. 6.7 AI Automation

MODULE 6 · SECTION 6.1

6.1 AI Agent Fundamentals

DEFINITION

> An AI agent is a system where a model decides what to do next: it plans, calls tools, observes results and repeats until the goal is met or it stops.

HOW IT WORKS

- A chatbot answers one message. An agent runs a loop: plan, act (tool call), observe, decide.
- Components: the model (reasoning), tools (actions such as search, database write, email), memory (state and history), and guardrails (limits and approvals).
- Autonomy must be bounded: set maximum steps, budgets and which actions need a human.

EVERY TOPIC EXPLAINED

What an agent is; chatbot vs agent; agent architecture. An AI agent uses a model to decide its own next steps toward a goal, calling tools and reacting to results. A chatbot replies to messages; an agent takes actions (search, update a CRM, send an email). Agent architecture = model + instructions + tools + memory/state + a control loop + guardrails.

Reasoning, planning, tools and actions, memory, state. Reasoning interprets the goal and the current situation. Planning breaks the goal into steps. Tools and actions change or read the outside world. Memory keeps short-term state (this task) and long-term facts (user preferences). State is the structured record of progress that the loop updates each step.

Autonomous vs supervised agents; limitations. Autonomous agents act without approval inside set limits; supervised agents pause for human approval at key points. Limitations: they can loop, misuse tools, follow injected instructions, run up costs or compound small errors, so production agents need step limits, budgets, logging and approvals.

WORKED EXAMPLE

> An order-status agent looks up an order (tool), sees it is delayed, drafts an apology, and asks a human to approve the refund it proposes.

RESOURCES

- Anthropic: building effective agents https://www.anthropic.com/engineering/building-effective-agents
- OpenAI: agents guide https://platform.openai.com/docs/guides/agents

6.2 Agent Development Frameworks

DEFINITION

> Frameworks give you the agent loop, tool wiring, state and tracing so you do not write them from scratch.

HOW IT WORKS

- LangGraph models an agent as a graph of steps with state, good for controllable workflows.
- LangChain provides model, tool and retriever components. CrewAI organises role-based agent teams. LlamaIndex focuses on data and RAG.
- OpenAI Agents SDK and Google's Agent Development Kit (ADK) provide agent loops tied to their providers.
- Choose the simplest option that meets the need; a plain function loop is often enough.

EVERY TOPIC EXPLAINED

LangChain, LangGraph, CrewAI, LlamaIndex. LangChain: components for models, prompts, tools, retrievers and agents, with many integrations. LangGraph: builds agents and workflows as graphs of nodes and edges with explicit state, checkpoints and human-in-the-loop pauses. CrewAI: defines agents with roles, goals and tasks that collaborate as a crew. LlamaIndex: data framework for ingestion, indexing and RAG, plus agents.

OpenAI Agents SDK, Google Agent Development Kit. OpenAI Agents SDK: lightweight Python/TypeScript library for agents, tools, hand-offs between agents, guardrails and tracing on OpenAI models. Google Agent Development Kit (ADK): framework for building, evaluating and deploying agents, integrated with Gemini and Vertex AI. Claude Agent SDK provides the agent loop used by Claude Code.

Framework comparison and selection. Compare frameworks on control (can you see and change every step?), state and persistence, tracing, provider support, community and maturity. Pick plain code for simple flows, LangGraph for complex stateful flows, CrewAI for quick role-based prototypes, LlamaIndex for data-heavy RAG.

WORKED EXAMPLE

> A claims-triage agent is built as a LangGraph graph: extract, check policy (tool), decide, and route to a human for amounts over a limit.

RESOURCES

- LangGraph documentation (official) https://docs.langchain.com/oss/langgraph/
- CrewAI docs https://docs.crewai.com/

6.3 Tool Calling

DEFINITION

> Tool calling lets a model request a named function with structured arguments; your code runs it and returns the result to the model.

HOW IT WORKS

- Each tool has a name, description and JSON schema for its inputs. The model returns a tool call; your code validates the arguments, runs the function, and sends back the result.
- Permissions: a tool can read freely but write only with checks. Validate every argument and never trust model output as safe SQL or shell.
- Handle errors: return a clear error message so the model can retry or stop.

EVERY TOPIC EXPLAINED

Function calling; tool schemas; API-based tools. Function calling: you describe functions to the model; it returns the function name and JSON arguments instead of text when a function is needed. Tool schemas use JSON Schema (name, description, parameters with types and required fields). API-based tools wrap external APIs (weather, CRM, payments) as callable functions.

Tool permissions, execution, validation and error handling. Tool permissions: decide which tools each agent may use and whether they read or write. Execution happens in your code, never inside the model. Validate arguments (types, ranges, allowed values) before execution. Error handling returns clear messages so the model can correct itself, with a maximum number of retries.

WORKED EXAMPLE

> A tool 'create_ticket(title, priority)' validates priority is one of low/medium/high, creates the ticket, and returns the new ticket ID.

RESOURCES

- Anthropic: tool use overview https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview
- OpenAI: function calling https://platform.openai.com/docs/guides/function-calling

6.4 MCP: Model Context Protocol

DEFINITION

> MCP is an open protocol that connects AI applications to external tools, data and prompts through standard servers, so one integration works in many apps.

HOW IT WORKS

- Architecture: a host (the AI app) creates one client per server; each server exposes resources (data), prompts and tools. Messages use JSON-RPC 2.0.
- Transports: STDIO for local servers; Streamable HTTP for remote servers.
- Security: servers need authentication, least-privilege scopes, and human consent for sensitive tools. The protocol version matters; check the spec version you target.

EVERY TOPIC EXPLAINED

MCP architecture; MCP servers and clients. MCP (Model Context Protocol) standardises how AI applications connect to tools and data. An MCP host (Claude Desktop, Claude Code, an IDE) runs MCP clients; each client connects to one MCP server; the server exposes capabilities over JSON-RPC 2.0, via STDIO (local) or Streamable HTTP (remote).

Resources, prompts and tools; authentication and security. Servers expose three primitives: resources (readable data such as files or records), prompts (reusable templates) and tools (actions the model can call). Remote servers use OAuth-based authorisation; apply least privilege, validate inputs, ask for user consent on sensitive tools, and only install servers you trust.

WORKED EXAMPLE

> A company builds one MCP server for its CRM; Claude, a support app and an internal agent all use the same tools with the same permissions.

RESOURCES

- Model Context Protocol: architecture (official) https://modelcontextprotocol.io/specification/2025-11-25/architecture
- MCP documentation (official) https://modelcontextprotocol.io/

6.5 Multi-Agent Systems

DEFINITION

> A multi-agent system uses several specialised agents that coordinate: one plans, others execute, and a checker reviews.

HOW IT WORKS

- Patterns: supervisor (one agent delegates), pipeline (each agent passes work on), and debate or review (agents check each other).
- Coordination needs shared state, clear hand-off formats, and a stop rule. More agents mean more cost and more ways to fail.
- Start with one agent and a few tools; add agents only when a single agent cannot keep the task straight.

EVERY TOPIC EXPLAINED

Single vs multi-agent architecture; delegation and communication. Single-agent: one model with several tools; simpler and easier to debug. Multi-agent: several specialised agents (researcher, writer, reviewer). Delegation passes a sub-task to another agent; communication happens through messages or shared state.

Supervisor agents, orchestration, workflow execution, state, failure recovery. A supervisor (orchestrator) agent plans and routes work to worker agents and combines results. Orchestration defines the order and conditions. Workflow execution may run steps in parallel. State management keeps one shared record of progress. Failure recovery retries a step, switches to a fallback or escalates to a human.

WORKED EXAMPLE

> A research system: a planner splits a report into five sections, five writer agents draft them, and a reviewer agent checks citations before assembly.

RESOURCES

- Anthropic: multi-agent research system https://www.anthropic.com/engineering/built-multi-agent-research-system
- LangGraph: multi-agent concepts https://docs.langchain.com/oss/langgraph/

6.6 Human-in-the-Loop

DEFINITION

> Human-in-the-loop means a person approves, corrects or takes over at defined points, especially for risky or costly actions.

HOW IT WORKS

- Define which actions need approval (payments, emails to customers, deletions) and set confidence or amount thresholds.
- Log every proposed and approved action with who approved it, so an audit trail exists.
- Escalate when the agent is unsure, fails repeatedly, or the request is outside its scope.

EVERY TOPIC EXPLAINED

Human approval, escalation, manual review, sensitive action approval. Human approval: the agent pauses before an action until a person approves. Escalation: hand the case to a human when confidence is low or rules require it. Manual review: a person checks a sample or all outputs. Sensitive action approval applies to payments, deletions, external emails and legal or medical content.

Audit trails, agent permissions, safe execution, production controls. Audit trails log who did what, when and why (prompts, tool calls, approvals). Agent permissions limit tools and data per agent. Safe execution uses sandboxes, dry-run modes and spending limits. Production controls include kill switches, rate limits, monitoring and alerting.

WORKED EXAMPLE

> An agent drafts a refund; any refund over $200 is sent to a manager's queue with the reasoning and the approve/reject buttons.

RESOURCES

- LangGraph: human-in-the-loop https://docs.langchain.com/oss/langgraph/interrupts
- Anthropic: building effective agents https://www.anthropic.com/engineering/building-effective-agents

6.7 AI Automation

DEFINITION

> AI automation connects triggers, data and steps across apps so a process runs without manual work, with an AI step where judgement is needed.

HOW IT WORKS

- Triggers start a run (a webhook, a schedule, a new row). Steps move data between tools through APIs.
- No-code tools (n8n, Zapier, Make) provide visual nodes; custom code is used when the logic is complex.
- Production automations need error handling, retries, idempotency (no duplicates on retry), monitoring and version control.

EVERY TOPIC EXPLAINED

n8n, Zapier, Make; webhooks; API integrations. n8n: open-source, self-hostable workflow automation with code nodes and AI agent nodes. Zapier: easiest no-code automation with thousands of app connectors. Make: visual scenarios with powerful data mapping. Webhooks start workflows from external events. API integrations connect any service that has an API.

CRM and email automation; WhatsApp Business integrations. CRM automation: create and update leads, deals and tasks (HubSpot, Salesforce, Zoho). Email automation: send, classify and reply to emails. WhatsApp Business integrations: use the official WhatsApp Business Platform (Cloud API) through Meta or a provider to send templates and reply to customers within policy rules.

Document processing; scheduled workflows; error handling; monitoring; versioning. Document processing workflows extract data from uploads. Scheduled workflows run on a timer (daily reports). Error handling uses error branches, retries and alerts. Monitoring tracks failed runs. Versioning keeps workflow exports in Git so changes can be reviewed and rolled back.

WORKED EXAMPLE

> A new WhatsApp enquiry triggers a workflow: the AI classifies intent, creates a CRM lead, sends a reply, and books a call if the lead is qualified.

RESOURCES

- n8n documentation (official) https://docs.n8n.io/
- Zapier help https://help.zapier.com/
- Make help center https://help.make.com/